Document Title

Author Name

September 11, 2026

Contents

1 Introduction

The utility of CompCert is that it provides a platform for reasoning formally about all the potential behaviours of compiled source-code. Given a piece of C source-code, the user must be able to infer (and reason about) all possible runtime-behaviours that could be produced upon executing the target program generated by compiling it. CompCertTSO is the first and to our knowledge only existing extension of CompCert that provides a formal model for reasoning about the behaviours of compiled source-code in a concurrent setting. Our intent is to extend CompCertTSO to accurately model RC11 concurrent behaviours, not TSO.

2 Design

In order to properly model this problem in Rocq, we must first be explicit about what we mean by program behaviour. As in CompCertTSO, we define the behaviour of a program to be the set of all possible observable behaviours that can be produced by executing the program. More precisely, we define observable behaviours to be traces of events that are produced by executing the program. Events are defined to be,

event ::= call fn_id args | return type value | exit code | fail

Notice that we do not include memory events in our definition of observable behaviours. Between compilation passes memory events may be introduced, removed or reordered. Therefore, we require a model of program behaviour that is independent of the precise memory traces that produce it. In this concurrent context, traces of events are generated by the interleaving of thread-local traces produced by multiple threads. As in CompCertTSO we decouple thread-local semantics, and therefore thread-local traces, from the memory model, allowing potentially non-sensical behaviours at the thread local level. We assert only at the whole system level that when a memory event is produced by a thread-local trace, it must be consistent with the memory model. This filters out all the non-sensical behaviours at the thread-local level in order to compose a consistent whole-system behaviour. Thread-local traces are defined by the following events,

thread-local-event ::= external event | mem mevent | τ | start tid p | exit

Memory traces are defined by the following events.

memory-event ::= read loc type value | write loc type value | rmw loc type value | fence | malloc loc type | free

3 Memory Model Semantics

Whereas in CompCertTSO, the behaviour of the memory model was defined by the TSO abstract machine, an operational model that uses ordered threadwise buffers to simulate the total store order semantics, we express the behaviour of the memory model as a set of axioms that decide upon the consistency of an execution graph generated by incrementally adding events and relations that satisfy these axioms. A memory event (node) and its corresponding primary edges (po, mo, rf) are added to the execution graph via a legal step in the memory model semantics. A legal step is defined as a step that does not violate any of the axioms of the RC11 memory model. We rely on the work of IMM to define the consistency of execution graphs. As part of the chain of proofs that define the correctness of the entire compiler, the consistency axioms of the memory model may change between compilation passes. In order to account for this, we define the memory model semantics to be parameterised by a consistency predicate that is defined by

the axioms of some specific memory model. IMM defines events to be reads, writes or fences, we must also account for the addition of malloc, free, thread creation and thread join events. To do this we interpret malloc and free events as writes to the location they allocate or free. Initially all locations are considered to be unallocated. Malloc events allocate a location with an undefined value. Free, on the other hand, deallocates a location and therefore removes it from the set of allocated locations (writes the unallocated value to the location). Importantly our operational semantics does not allow malloc to allocate the same location twice, ie. calls to malloc always return an event with a fresh location. This is an important constraint that is not enforced by the memory model, but is necessary to ensure that our operational semantics are well-defined. Allowing malloc to allocate the same location twice, raises the question of when can an event be considered unallocated and hence available to be re-allocated. This question is beyond the scope of this work, but we note that it is an important question that must be answered in order to define a realistic operational semantics for malloc and free, since allocators re-use locations. To model thread creation and join events, we reinterpret IMM's notion of ordering between threads and events. Both threads and events possess an identifier, the composition of which is unique. IMM models these identifiers simply as natural numbers as it provides no ordering between threads expressing their provenance. In order for us to model thread creation and join we expand the expressivity of said identifiers such that events and threads form an ordering. Specifically both tids and uids idependently form a lattice structure such that,

$$\forall tid_1, tid_2 \in \text{TID}.\ \exists tid_3, tid_4 \in \text{TID}.\ \text{s.t.}\ tid_1 \sqcap tid_2 = tid_3 \wedge tid_1 \sqcup tid_2 = tid_4$$

and

$$\forall uid_1, uid_2 \in \text{UID}.\ \exists uid_3, uid_4 \in \text{UID}.\ \text{s.t.}\ uid_1 \sqcap uid_2 = uid_3 \wedge uid_1 \sqcup uid_2 = uid_4$$

The idea is that the meet of two threads is their greatest common ancestor and their join is their least common successor, this allows us to express the ext_sb relation in IMM as follows,

```
Definition ext_sb (x y : event) : Prop :=
    (tid x <= tid y) /\ (if tid x = tid y then uid x < uid y else True)
.
```

The small step semantics of the memory model is defined by the following set of inference rules, that define the conditions under which a memory event can be added to an execution graph. Execution graphs are denoted using Ψ, δ denotes the newly added event and Ψ' denotes the new execution graph after the addition of δ. Functions over execution graphs are denoted using the subscript Ψ, functions that are independent of the execution graph are denoted without a subscript. E_Ψ is used to denote the set of events in an execution graph, V_Ψ is used to denote the set of edges within an execution graph. The consistency predicate $\text{Consistent}_M(\Psi)$ is used to denote that the execution graph Ψ satisfies the axioms of some memory model M. The isAllocated_Ψ predicate is defined to be true if the value of the terminal write event to the location of the newly added event is not the unallocated value.

$$\text{isAllocated}_\Psi\ \text{x}\ := \text{val}(\text{last}_\Psi(x)) \neq \text{VUnalloc}$$

We note that it should come as a lemma that it is decidable whether a location is allocated or not, since the execution graph is finite and the initial write is always present.

For the MMalloc rule we need a specific predicate deciding whether or not a location has ever been allocatd. The $\text{neverAllocated}_\Psi$, is defined to be true if the only event on that location is the initial unallocated write.

$$\text{neverAllocated}_\Psi\ \text{x}\ := \nexists \sigma \in E_\Psi\ \text{s.t.}\ (W_{init}\ \text{x}\ ; mo; \sigma) \in V_\Psi$$

$$\frac{\begin{array}{c}\text{Consistent}_M(\Psi) \\ \text{Consistent}_M(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{R x type v} \\ v \neq \text{VUnalloc}\end{array}}{\Psi \xrightarrow{\text{MRead x type v}} \Psi'} \text{MRead}$$

$$\frac{\begin{array}{c}\text{Consistent}_M(\Psi) \\ \text{Consistent}_M(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{W x type v} \\ \text{isAllocated}_\Psi\ \text{x}\end{array}}{\Psi \xrightarrow{\text{MWrite x type v}} \Psi'} \text{MWrite}$$

$$\frac{\begin{array}{c}\text{Consistent}_M(\Psi) \\ \text{Consistent}_M(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{R x type VUnalloc} \\ \neg(\text{isAllocated}_\Psi\ \text{x})\end{array}}{\Psi \xrightarrow{\text{MReadFail x}} \Psi'} \text{MReadFail}$$

$$\frac{\begin{array}{c}\text{Consistent}_M(\Psi) \\ \text{Consistent}_M(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{W x type v} \\ \neg(\text{isAllocated}_\Psi\ \text{x})\end{array}}{\Psi \xrightarrow{\text{MWriteFail x}} \Psi'} \text{MWriteFail}$$

$$\frac{\begin{array}{c}\text{Consistent}_M(\Psi) \\ \text{Consistent}_M(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{W x type VUndef} \\ \text{neverAllocated}_\Psi\ \text{x}\end{array}}{\Psi \xrightarrow{\text{MMalloc x}} \Psi'} \text{MMalloc}$$

$$\frac{\begin{array}{c}\text{Consistent}_M(\Psi) \\ \text{Consistent}_M(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{W x type VUnalloc} \\ \text{isAllocated}_\Psi\ \text{x}\end{array}}{\Psi \xrightarrow{\text{MFree x}} \Psi'} \text{MFree}$$

$$\frac{\begin{array}{c}\text{Consistent}_M(\Psi) \\ \text{Consistent}_M(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{R } t_1 \text{ type } vt_1 \\ \text{isAllocated}_\Psi\ t_1\end{array} \qquad \begin{array}{c}\text{tid}(\delta) = tid \\ \text{uid}(\delta) = \max\_\text{uid}(tid) \sqcup \max\_\text{uid}(tid')\end{array}}{\Psi \xrightarrow{\text{MJoin tid } t_1\ vt_1 \text{ tid'}} \Psi'} \text{MJoin}$$

3.1 Thread Creation and Joining

The above rules are purposefully missing the rule for thread creation. Since the intention is to use IMM definition of execution graphs, it will be necessary to alter the current definition to allow for arbitrarily nested thread lineage. IMM in its current state doesn't allow for thread lineage, all threads are equal and independent of each other. In order to accurately express the semantics of thread creation, we need to add an ordering between threads that reflects the parent-child relationship between threads. This is necessary to ensure that the memory events of a child thread are not visible to its parent thread until the child thread has been created. Furthermore it is essential to ensure that the completion of thread creation synchronizes-with the beginning of execution of the new thread as per the C standard.

I propose adding a partial-ordering on threads within IMM modelled as a lattice, where the meet of two threads is their common ancestor and their join is their least common successor. We may then redefine the sequenced-before utilising this ordering on threads, extending its definition to include events from different threads related by the thread ordering. Thread creation can then be modelled in the operational semantics by creating two new child threads, one representing the continuation of the parent thread and the other representing the new child thread. Both the newly created threads will be automatically synchronised with their parent, since there is a po edge from the last event of the parent thread to the first event of the child thread, but not with each other. Dually, the joining of threads can be modelled by synchronising the continuation of the parent thread with the completion of the child thread, ie. there is a po edge from the last event of the child thread to the join event of the parent thread.

It is important to note that both creation and joining of threads create logically new threads to model the continuation of the parent thread after the creation or joining event.

The *ext_sb* relation in IMM, from which the *po* relation is derived would be defined as follows,

$$\forall(\theta,\gamma) \in E_\Psi \;\; \theta; \text{ext\_sb}; \gamma \iff tid(\theta) \leq tid(\gamma) \wedge (\text{if } tid(\theta) = tid(\gamma) \text{ then } uid(\theta) < uid(\gamma))$$

3.2 Memory Model Lemmas

For each of the rules that add a read event, the following predicate is satisfied as a consequence of RC11 consistency and the definition of $E_{\Psi'}$.

$$\exists(\theta,\gamma) \in E_\Psi \text{ s.t. } \gamma = \text{W x type v} \wedge V_{\Psi'} = V_\Psi \cup \{\theta; \text{po}; \delta, \gamma; \text{rf}; \delta\}$$

Similarly, for each rule that adds a write event, the following predicate is satisfied as a consequence of RC11 consistency and the definition of $E_{Psi'}$.

$$\exists(\theta,\gamma) \in E_\Psi \text{ s.t. is\_write}(\gamma) \wedge V_{\Psi'} = V_\Psi \cup \{\theta; \text{po}; \delta, \gamma; \text{mo}; \delta\}$$

The operational memory model LTS parameterised by the consistency axiom M is then composed with the thread-local LTS, parameterised by a thread-local small-step semantics S, to define the whole-system semantics of a program. The small-step semantics of the whole-system is defined by the following inference rules, that define the conditions under which a thread-local step can be added to a whole-system trace.

$$\frac{\Sigma_S \xrightarrow{\text{TERead x type v}} \Sigma'_S \qquad \Psi_M \xrightarrow{\text{MRead x type v}} \Psi'_M}{\Psi_M \,;\, \Sigma_S \xrightarrow{\tau} \Psi'_M \,;\, \Sigma'_S}\ \textsc{Read}$$

$$\frac{\Sigma_S \xrightarrow{\text{TEWrite x type v}} \Sigma'_S \qquad \Psi_M \xrightarrow{\text{MWrite x type v}} \Psi'_M}{\Psi_M \,;\, \Sigma_S \xrightarrow{\tau} \Psi'_M \,;\, \Sigma'_S}\ \textsc{Write}$$

$$\frac{\Sigma_S \xrightarrow{\text{TERead x type v}} \Sigma'_S \qquad \Psi_M \xrightarrow{\text{MReadFail x}} \Psi'_M}{\Psi_M \,;\, \Sigma_S \xrightarrow{\text{Fail}} \Psi'_M \,;\, \Sigma'_S}\ \textsc{ReadFail}$$

$$\frac{\Sigma_S \xrightarrow{\text{TEWrite x type v}} \Sigma'_S \qquad \Psi_M \xrightarrow{\text{MWriteFail x}} \Psi'_M}{\Psi_M \,;\, \Sigma_S \xrightarrow{\text{Fail}} \Psi'_M \,;\, \Sigma'_S}\ \textsc{WriteFail}$$

$$\frac{\Sigma_S \xrightarrow{\text{TEJoin tid t vt tid'}} \Sigma'_S \qquad \Psi_M \xrightarrow{\text{MJoin tid t vt tid'}} \Psi'_M \qquad \Phi(vt) = \text{tid'}}{\Psi_M \,;\, \Sigma_S \xrightarrow{\tau} \Psi'_M \,;\, \Sigma'_S}\ \textsc{Join}$$

Traces generated by the whole-system semantics are then further filtered to produce the observable behaviours of a program. Specifically this ensures that all traces containing the fail event terminate at the point of failure and do not continue executing as is permitted by the whole-system semantics. The observable behaviours of a program are defined by the following set of traces, where ℓ is a finite sequence of events, tr is a trace and $args$ is a sequence of arguments. For now we do not permit programs that exhibit OOM behaviour, but this could possibly be added in the future.

$$\begin{aligned}
\text{traces}(p, args) \stackrel{\text{def}}{=} &\{\ell \cdot \text{end} \mid \exists s \in \text{init}(p, args).\ \exists s'.\ s \stackrel{\ell}{\Rightarrow} s' \not\rightarrow\} \\
&\cup \{\ell \cdot \text{inftau} \mid \exists s \in \text{init}(p, args).\ \exists s'.\ s \stackrel{\ell}{\Rightarrow} s' \xrightarrow{\tau^\omega}\} \\
&\cup \{\ell \cdot tr \mid \exists s \in \text{init}(p, args).\ \exists s'.\ s \stackrel{\ell}{\Rightarrow} s' \xrightarrow{\texttt{fail}}\} \\
&\cup \{l \mid \exists s \in \text{init}(p, args).\ s \stackrel{l}{\Rightarrow} \text{ and } l \text{ is infinite}\}
\end{aligned}$$

I conjecture that, given a proof that a memory model M_1 is a refinement of another memory model M_2 (ie. the same core execution graph can be interpreted under model M_1 as displaying a subsetof M_2's behaviours), then the whole-system semantics of a program with thread-local semantics S under M_1 is a refinement of the whole-system semantics of the same program under M_2, and therefore there is a trace-inclusion relation between the observable behaviours of a program under M_1 and the observable behaviours of the same program under M_2, satisfying the requirements of a compiler correctness proof. More formally,

$$\forall M_1, M_2, S, x \text{ s.t. } M_1 \sqsubseteq M_2 \implies \Psi_{M_1} \,;\, \Sigma_S \xrightarrow{x} \Psi'_{M_1} \,;\, \Sigma'_S \implies \Psi_{M_2} \,;\, \Sigma_S \xrightarrow{x} \Psi'_{M_2} \,;\, \Sigma'_S$$

We note that so far we're assuming an identity mapping between the memory events of the two memory models, but this may not be the case in general (see specifically IMM to Arm). In any case it suffices for RC11 to IMM and as a simple corrollary of the above general result we can prove that.

$$\forall M_1, M_2, S, t \text{ s.t. } M_1 \sqsubseteq M_2 \implies t^S_{M_1} \implies t^S_{M_2}$$

and hence that

$$\forall M_1, M_2, S, p, args \text{ s.t. } M_1 \sqsubseteq M_2 \implies \text{traces}^S_{M_1}(p, args) \subseteq \text{traces}^S_{M_2}(p, args)$$

4 Results

Present your results here.

5 Conclusion

Summarize your conclusions here.